

Don't be surprised if the Hindi text looks different in various consoles. By the way, try changing the TTS language to English and listen to the difference.

Read aloud application

You can have the computer read aloud one of the first nursery rhymes that I learnt. Save it in a text file, *NurseryRhyme.txt*.

```
माछी पाल की लगी है,  
जीवन समझा है पाछी,  
शाय समझो, इत पाछी,  
बाहर निकालो, मत जलूँगी।
```

Your basic program, *read_aloud.py*, would look like the following code:

```
import speechd
s = speechd.Speaker('ReadAloud')
s.set_output_module('espeak')
s.set_language('hi')
f = open('NurseryRhyme.txt')
s.set_rate(-50)
for line in f.readlines():
    sentence = unicode(line, 'utf8')
    print sentence
    s.speak(sentence)
s.close()
```

In Python 3, all strings will be Unicode, but currently, you will need to interpret the data read as a Unicode string.

You would notice that the entire poem is displayed even before the first line is spoken, but you want to display each line as it is being spoken. So, you need to think in terms of events. Your program needs to wait till the TTS engine has finished speaking the line. You would notice that the 'speak method' accepts a call back parameter and an events parameter. This ensures that the speech dispatcher will call your program back after the events you have requested.

The call-back method will be passed a parameter—the event, which resulted in it being called. The speech dispatcher currently has two events: *begin* and *end*. Needless to say, you would be interested only in the second event.

Python's threading module contains a class *Event* that will be very useful for this application. So the better version of your read aloud application will be what's shown below:

```
import speechd
from threading import Event
def spoken(event):
    speech_over.set()
s = speechd.Speaker('ReadAloud')
```

```
s.set_output_module('espeak')
s.set_language('hi')
f = open('NurseryRhyme.txt')
s.set_rate(-50)
speech_over = Event()
for line in f.readlines():
    sentence = unicode(line, 'utf8')
    print sentence
    s.speak(sentence, callback=spoken, event_types='end')
    speech_over.wait()
    speech_over.clear()
s.close()
```

You have added just six lines, modified one line and gained quite a bit of control over the ability to integrate the playing of sound with the rest of the application.

Dhvani and Festival

Dhvani won the FOSS India Award last year and is available at dhvani.sourceforge.net. Using the source code from the repository and with a little help from Santhosh Thottingal, I could try the above examples of code. The voice quality seems more natural than the eSpeak voice. I hope the package evolves and becomes a part of the Fedora and Ubuntu repositories.

Information about integration with a speech dispatcher is available at dhvani.sourceforge.net/doc/screenreader.html. In our Python code, the change needed is:

```
s.set_output_module('dhvani-generic')
```

Debian and Ubuntu have a package for a Hindi voice for Festival. The quality of voice is acceptable but eSpeak's is better.

If more applications used TTS, the speech quality would definitely improve. If the speech quality was better, more applications would have used TTS.

Fortunately, smart cell phones, Netbooks and ebook readers are going to change the dynamics of the current roadblock. Adding language support to the eSpeak project is likely to get acceptance the fastest. (On how to go about it, check out: http://espeak.sourceforge.net/add_language.html)

If user applications rely on a speech dispatcher as Python encourages us to do, it is easier to use a TTS engine optimised for specific languages, e.g., Dhvani for Indic scripts. It is just a configuration detail.

It is worth adding TTS to your applications today, for no reason other than to attract attention. 

By: Dr. Anil Seth

The author is a consultant by profession and can be reached at seth.anil@gmail.com